

10 Logistic Regression

10.1 Introduction

Logistic regression is a widely used discriminative classification model $p(y|\mathbf{x}; \boldsymbol{\theta})$, where $\mathbf{x} \in \mathbb{R}^D$ is a fixed-dimensional input vector, $y \in \{1, \dots, C\}$ is the class label, and $\boldsymbol{\theta}$ are the parameters. If $C = 2$, this is known as **binary logistic regression**, and if $C > 2$, it is known as **multinomial logistic regression**, or alternatively, **multiclass logistic regression**. We give the details below.

10.2 Binary logistic regression

Binary logistic regression corresponds to the following model

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x} + b)) \quad (10.1)$$

where σ is the sigmoid function defined in Section 2.4.2, $\mathbf{w}$ are the weights, b is the bias, and $\boldsymbol{\theta} = (\mathbf{w}, b)$ are all the parameters. In other words,

$$p(y = 1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}} \quad (10.2)$$

where $a = \mathbf{w}^\top \mathbf{x} + b = \log(\frac{p}{1-p})$, is the log-odds, and $p = p(y = 1|\mathbf{x}, \boldsymbol{\theta})$. (In ML, the quantity a is usually called the **logit** or the **pre-activation**.)

Sometimes we choose to use the labels $\tilde{y} \in \{-1, +1\}$ instead of $y \in \{0, 1\}$. We can compute the probability of these alternative labels using

$$p(\tilde{y}|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\tilde{y}a) \quad (10.3)$$

since $\sigma(-a) = 1 - \sigma(a)$. This slightly more compact notation is widely used in the ML literature.

10.2.1 Linear classifiers

The sigmoid gives the probability that the class label is $y = 1$. If the loss for misclassifying each class is the same, then the optimal decision rule is to predict $y = 1$ iff class 1 is more likely than class 0, as we explained in Section 5.1.2.2. Thus

$$f(\mathbf{x}) = \mathbb{I}(p(y = 1|\mathbf{x}) > p(y = 0|\mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1|\mathbf{x})}{p(y = 0|\mathbf{x})} > 0\right) = \mathbb{I}(a > 0) \quad (10.4)$$

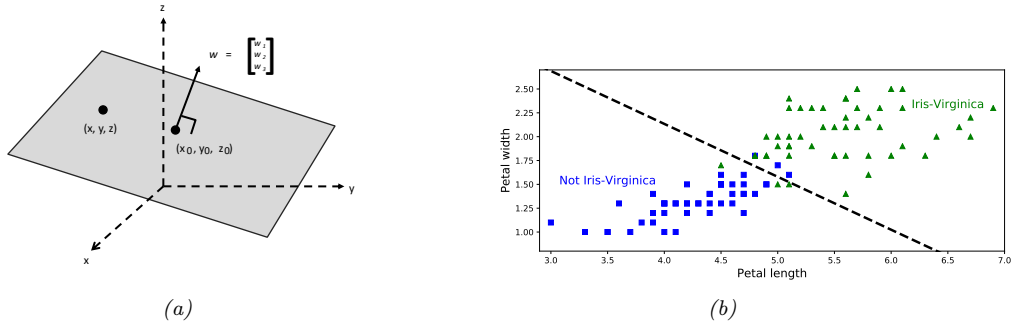


Figure 10.1: (a) Visualization of a 2d plane in a 3d space with surface normal $\mathbf{w}$ going through point $\mathbf{x}_0 = (x_0, y_0, z_0)$. See text for details. (b) Visualization of optimal linear decision boundary induced by logistic regression on a 2-class, 2-feature version of the iris dataset. Generated by [iris_logreg.ipynb](#). Adapted from Figure 4.24 of [Gér19].

where $a = \mathbf{w}^\top \mathbf{x} + b$.

Thus we can write the prediction function as follows:

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^\top \mathbf{x} = b + \sum_{d=1}^D w_d x_d \quad (10.5)$$

where $\mathbf{w}^\top \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle$ is the inner product between the weight vector $\mathbf{w}$ and the feature vector $\mathbf{x}$. This function defines a linear **hyperplane**, with normal vector $\mathbf{w} \in \mathbb{R}^D$ and an offset $b \in \mathbb{R}$ from the origin.

Equation (10.5) can be understood by looking at Figure 10.1a. Here we show a plane in a 3d feature space going through the point $\mathbf{x}_0$ with surface normal $\mathbf{w}$. Points on the surface satisfy $\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_0) = 0$. If we define $b = -\mathbf{w}^\top \mathbf{x}_0$, we can rewrite this as $\mathbf{w}^\top \mathbf{x} + b = 0$. This plane separates 3d space into two **half spaces**. This linear plane is known as a **decision boundary**. If we can perfectly separate the training examples by such a linear boundary (without making any classification errors on the training set), we say the data is **linearly separable**. From Figure 10.1b, we see that the two-class, two-feature version of the iris dataset is not linearly separable.

In general, there will be uncertainty about the correct class label, so we need to predict a probability distribution over labels, and not just decide which side of the decision boundary we are on. In Figure 10.2, we plot $p(y = 1 | (x_1, x_2), \mathbf{w}) = \sigma(w_1 x_1 + w_2 x_2)$ for different weight vectors $\mathbf{w}$. The vector $\mathbf{w}$ defines the orientation of the decision boundary, and its magnitude, $\|\mathbf{w}\| = \sqrt{\sum_{d=1}^D w_d^2}$, controls the steepness of the sigmoid, and hence the confidence of the predictions.

10.2.2 Nonlinear classifiers

We can often make a problem linearly separable by preprocessing the inputs in a suitable way. In particular, let $\phi(\mathbf{x})$ be a transformed version of the input feature vector. For example, suppose we use $\phi(x_1, x_2) = [1, x_1^2, x_2^2]$, and we let $\mathbf{w} = [-R^2, 1, 1]$. Then $\mathbf{w}^\top \phi(\mathbf{x}) = x_1^2 + x_2^2 - R^2$, so the decision boundary (where $f(\mathbf{x}) = 0$) defines a circle with radius R , as shown in Figure 10.3. The resulting

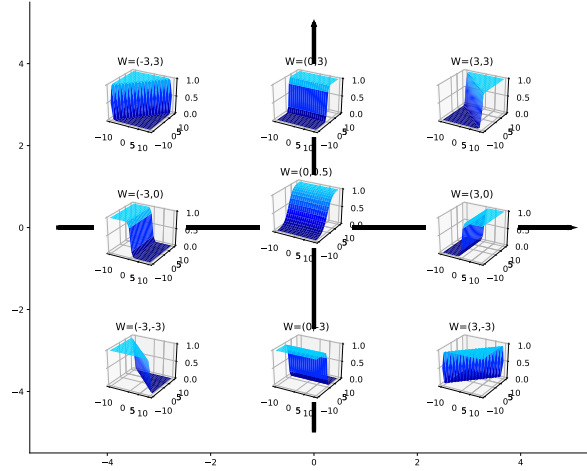


Figure 10.2: Plots of $\sigma(w_1x_1 + w_2x_2)$. Here $\mathbf{w} = (w_1, w_2)$ defines the normal to the decision boundary. Points to the right of this have $\sigma(\mathbf{w}^\top \mathbf{x}) > 0.5$, and points to the left have $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$. Adapted from Figure 39.3 of [Mac03]. Generated by `sigmoid_2d_plot.ipynb`.

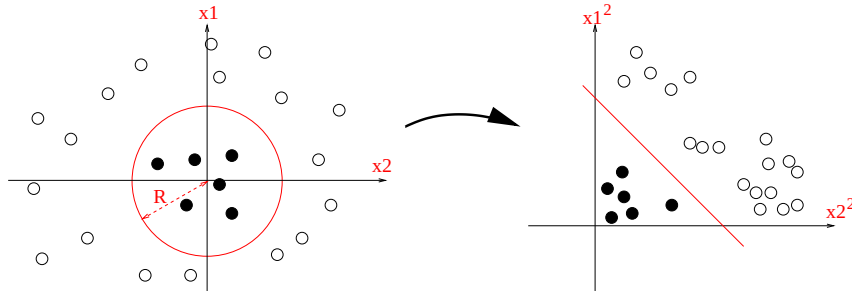


Figure 10.3: Illustration of how we can transform a quadratic decision boundary into a linear one by transforming the features from $\mathbf{x} = (x_1, x_2)$ to $\phi(\mathbf{x}) = (x_1^2, x_2^2)$. Used with kind permission of Jean-Philippe Vert.

function f is still linear in the parameters $\mathbf{w}$, which is important for simplifying the learning problem, as we will see in Section 10.2.3. However, we can gain even more power by learning the parameters of the feature extractor $\phi(\mathbf{x})$ in addition to linear weights $\mathbf{w}$; we discuss how to do this in Part III.

In Figure 10.3, we used a quadratic expansion of the features. We can also use a higher order polynomial, as in Section 1.2.2.2. In Figure 10.4, we show the effects of using polynomial expansion up to degree K on a 2d logistic regression problem. As in Figure 1.7, we see that the model becomes more complex as the number of parameters increases, and eventually results in overfitting. We discuss ways to reduce overfitting in Section 10.2.7.

10.2.3 Maximum likelihood estimation

In this section, we discuss how to estimate the parameters of a logistic regression model using maximum likelihood estimation.

10.2.3.1 Objective function

The negative log likelihood (scaled by the dataset size N) is given by the following (we assume the bias term b is absorbed into the weight vector $\mathbf{w}$):

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \log p(\mathcal{D}|\mathbf{w}) = -\frac{1}{N} \log \prod_{n=1}^N \text{Ber}(y_n|\mu_n) \quad (10.6)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log[\mu_n^{y_n} \times (1 - \mu_n)^{1-y_n}] \quad (10.7)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] \quad (10.8)$$

$$= \frac{1}{N} \sum_{n=1}^N \mathbb{H}_{ce}(y_n, \mu_n) \quad (10.9)$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the **logit**, and $\mathbb{H}_{ce}(y_n, \mu_n)$ is the **binary cross entropy** defined by

$$\mathbb{H}_{ce}(p, q) = -[p \log q + (1 - p) \log(1 - q)] \quad (10.10)$$

If we use $\tilde{y}_n \in \{-1, +1\}$ instead of $y_n \in \{0, 1\}$, then we can rewrite this as follows:

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [\mathbb{I}(\tilde{y}_n = 1) \log(\sigma(a_n)) + \mathbb{I}(\tilde{y}_n = -1) \log(\sigma(-a_n))] \quad (10.11)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log(\sigma(\tilde{y}_n a_n)) \quad (10.12)$$

$$= \frac{1}{N} \sum_{n=1}^N \log(1 + \exp(-\tilde{y}_n a_n)) \quad (10.13)$$

However, in this book, we will mostly use the $y_n \in \{0, 1\}$ notation, since it is easier to generalize to the multiclass case (Section 10.3), and makes the connection with cross-entropy easier to see.

10.2.3.2 Optimizing the objective

To find the MLE, we must solve

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) = \mathbf{0} \quad (10.14)$$

We can use any gradient-based optimization algorithm to solve this, such as those we discuss in Chapter 8. We give a specific example in Section 10.2.4. But first we must derive the gradient, as we explain below.

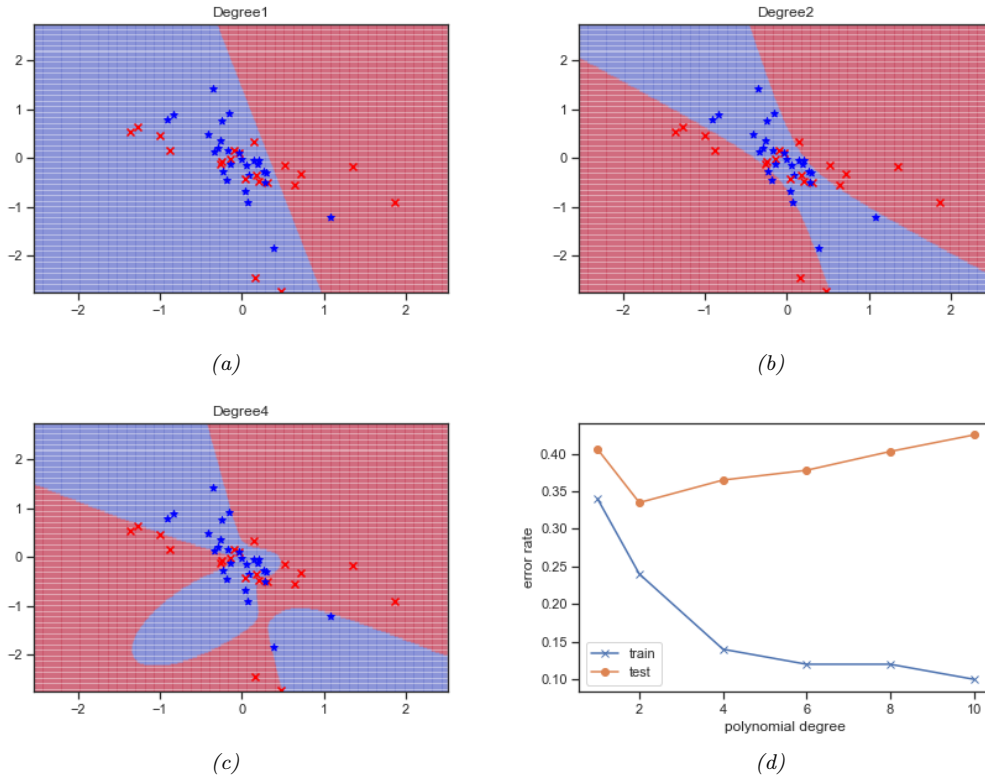


Figure 10.4: Polynomial feature expansion applied to a two-class, two-dimensional logistic regression problem. (a) Degree $K = 1$. (b) Degree $K = 2$. (c) Degree $K = 4$. (d) Train and test error vs degree. Generated by [logreg_poly_demo.ipynb](#).

10.2.3.3 Deriving the gradient

Although we can use automatic differentiation methods (Section 13.3) to compute the gradient of the NLL, it is also easy to do explicitly, as we show below. Fortunately the resulting equations will turn out to have a simple and intuitive interpretation, which can be used to derive other methods, as we will see.

To start, note that

$$\frac{d\mu_n}{da_n} = \sigma(a_n)(1 - \sigma(a_n)) \quad (10.15)$$

where $a_n = \mathbf{w}^\top \mathbf{x}_n$ and $\mu_n = \sigma(a_n)$. Hence by the chain rule (and the rules of vector calculus, discussed in Section 7.8) we have

$$\frac{\partial}{\partial w_d} \mu_n = \frac{\partial}{\partial w_d} \sigma(\mathbf{w}^\top \mathbf{x}_n) = \frac{\partial}{\partial a_n} \sigma(a_n) \frac{\partial a_n}{\partial w_d} = \mu_n(1 - \mu_n)x_{nd} \quad (10.16)$$

The gradient for the bias term can be derived in the same way, by using the input $x_{n0} = 1$ in the above equation. However, we will ignore the bias term for simplicity. Hence

$$\nabla_{\mathbf{w}} \log(\mu_n) = \frac{1}{\mu_n} \nabla_{\mathbf{w}} \mu_n = (1 - \mu_n) \mathbf{x}_n \quad (10.17)$$

Similarly,

$$\nabla_{\mathbf{w}} \log(1 - \mu_n) = \frac{-\mu_n(1 - \mu_n) \mathbf{x}_n}{1 - \mu_n} = -\mu_n \mathbf{x}_n \quad (10.18)$$

Thus the gradient vector of the NLL is given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n(1 - \mu_n) \mathbf{x}_n - (1 - y_n) \mu_n \mathbf{x}_n] \quad (10.19)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \mathbf{x}_n - y_n \mathbf{x}_n \mu_n - \mathbf{x}_n \mu_n + y_n \mathbf{x}_n \mu_n] \quad (10.20)$$

$$= \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n \quad (10.21)$$

If we interpret $e_n = \mu_n - y_n$ as an error signal, we can see that the gradient weights each input $\mathbf{x}_n$ by its error, and then averages the result. Note that we can rewrite the gradient in matrix form as follows:

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} (\mathbf{1}_N^\top (\text{diag}(\boldsymbol{\mu} - \mathbf{y}) \mathbf{X}))^\top \quad (10.22)$$

where $\mathbf{X}$ is the $N \times D$ design matrix containing the examples $\mathbf{x}_n$ in each row.

10.2.3.4 Deriving the Hessian

Gradient-based optimizers will find a stationary point where $\mathbf{g}(\mathbf{w}) = \mathbf{0}$. This could either be a global optimum or a local optimum. To be sure the stationary point is the global optimum, we must show that the objective is **convex**, for reasons we explain in Section 8.1.1.1. Intuitively this means that the NLL has a **bowl shape**, with a unique lowest point, which is indeed the case, as illustrated in Figure 10.5b.

More formally, we must prove that the Hessian is positive semi-definite, which we now do. (See Chapter 7 for relevant background information on linear algebra.) One can show that the Hessian is given by

$$\mathbf{H}(\mathbf{w}) = \nabla_{\mathbf{w}} \nabla_{\mathbf{w}}^\top \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n(1 - \mu_n) \mathbf{x}_n) \mathbf{x}_n^\top = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X} \quad (10.23)$$

where

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)) \quad (10.24)$$

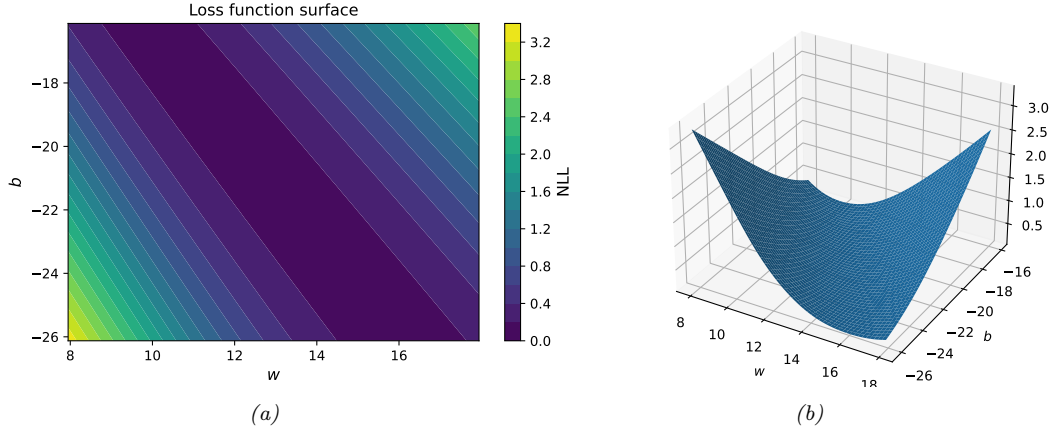


Figure 10.5: NLL loss surface for binary logistic regression applied to *Iris* dataset with 1 feature and 1 bias term. The goal is to minimize the function. The global MLE is at the center of the plot. Generated by [iris_logreg_loss_surface.ipynb](#).

We see that $\mathbf{H}$ is positive definite, since for any nonzero vector $\mathbf{v}$, we have

$$\mathbf{v}^\top \mathbf{X}^\top \mathbf{S} \mathbf{X} \mathbf{v} = (\mathbf{v}^\top \mathbf{X}^\top \mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v}) = \|\mathbf{v}^\top \mathbf{X}^\top \mathbf{S}^{\frac{1}{2}}\|_2^2 > 0 \quad (10.25)$$

This follows since $\mu_n > 0$ for all n , because of the use of the sigmoid function. Consequently the NLL is strictly convex. However, in practice, values of μ_n which are close to 0 or 1 might cause the Hessian to be close to singular. We can avoid this by using ℓ_2 regularization, as we discuss in Section 10.2.7.

10.2.4 Stochastic gradient descent

Our goal is to solve the following optimization problem

$$\hat{\mathbf{w}} \triangleq \underset{\mathbf{w}}{\operatorname{argmin}} \mathcal{L}(\mathbf{w}) \quad (10.26)$$

where $\mathcal{L}(\mathbf{w})$ is the loss function, in this case the negative log likelihood:

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] \quad (10.27)$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, and $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the log odds.

There are many algorithms we could use to solve Equation (10.26), as we discuss in Chapter 8. Perhaps the simplest is to use stochastic gradient descent (Section 8.4). If we use a minibatch of size 1, then we get the following simple update equation:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}_t) = \mathbf{w}_t - \eta_t (\mu_n - y_n) \mathbf{x}_n \quad (10.28)$$

where we replaced the average over all N examples in the gradient of Equation (10.21) with a single stochastically chosen sample n . (The index n changes with t .)

Since we know the objective is convex (see Section 10.2.3.4), then one can show that this procedure will converge to the global optimum, provided we decay the learning rate at the appropriate rate (see Section 8.4.3). We can improve the convergence speed using variance reduction techniques such as SAGA (Section 8.4.5.2).

10.2.5 Perceptron algorithm

A **perceptron**, first introduced in [Ros58], is a deterministic binary classifier of the following form:

$$f(\mathbf{x}_n; \boldsymbol{\theta}) = \mathbb{I}(\mathbf{w}^\top \mathbf{x}_n + b > 0) \quad (10.29)$$

This can be seen to be a limiting case of a binary logistic regression classifier, in which the sigmoid function $\sigma(a)$ is replaced by the Heaviside step function $H(a) \triangleq \mathbb{I}(a > 0)$. See Figure 2.10 for a comparison of these two functions.

Since the Heaviside function is not differentiable, we cannot use gradient-based optimization methods to fit this model. However, Rosenblatt proposed the **perceptron learning algorithm** instead. The basic idea is to start with random weights, and then iteratively update them whenever the model makes a prediction mistake. More precisely, we update the weights using

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\hat{y}_n - y_n)\mathbf{x}_n \quad (10.30)$$

where $(\mathbf{x}_n, y_n)$ is the labeled example sampled at iteration t , and η_t is the learning rate or step size. (We can set the step size to 1, since the magnitude of the weights does not affect the decision boundary.) See [perceptron_demo_2d.ipynb](#) for a simple implementation of this algorithm.

The perceptron update rule in Equation (10.30) has an intuitive interpretation: if the prediction is correct, no change is made, otherwise we move the weights in a direction so as to make the correct answer more likely. More precisely, if $y_n = 1$ and $\hat{y}_n = 0$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_n$, and if $y_n = 0$ and $\hat{y}_n = 1$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}_n$.

By comparing Equation (10.30) to Equation (10.28), we see that the perceptron update rule is equivalent to the SGD update rule for binary logistic regression using the approximation where we replace the soft probabilities $\mu_n = p(y_n = 1 | \mathbf{x}_n)$ with hard labels $\hat{y}_n = f(\mathbf{x}_n)$. The advantage of the perceptron method is that we don't need to compute probabilities, which can be useful when the label space is very large. The disadvantage is that the method will only converge when the data is linearly separable [Nov62], whereas SGD for minimizing the NLL for logistic regression will always converge to the globally optimal MLE, even if the data is not linearly separable.

In Section 13.2, we will generalize perceptrons to nonlinear functions, thus significantly enhancing their usefulness.

10.2.6 Iteratively reweighted least squares

Gradient descent is a **first order** optimization method, which means it only uses first order gradients to navigate through the loss landscape. This can be slow, especially when some directions of space point steeply downhill, whereas other have a shallower gradient, as is the case in Figure 10.5a. In such problems, it can be much faster to use a **second order** optimization method, that takes the curvature of the space into account.

We discuss such methods in more detail in Section 8.3. Here we just consider a simple second order method that works well for logistic regression. We focus on the full batch setting (so we assume N is small), since it is harder to make second order methods work in the stochastic setting (see e.g., [Byr+16; Liu+18b] for some methods).

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{H}_t^{-1} \mathbf{g}_t \quad (10.31)$$

where

$$\mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\mathbf{w})|_{\mathbf{w}_t} = \nabla^2 \mathcal{L}(\mathbf{w}_t) = \mathbf{H}(\mathbf{w}_t) \quad (10.32)$$

is assumed to be positive-definite to ensure the update is well-defined. If the Hessian is exact, we can set the step size to $\eta_t = 1$.

We now apply this method to logistic regression. Recall from Section 10.2.3.3 that the gradient and Hessian are given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n \quad (10.33)$$

$$\mathbf{H} = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X} \quad (10.34)$$

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)) \quad (10.35)$$

Hence the Newton update has the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1} \mathbf{g}_t \quad (10.36)$$

$$= \mathbf{w}_t + (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t) \quad (10.37)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} [(\mathbf{X}^\top \mathbf{S}_t \mathbf{X}) \mathbf{w}_t + \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t)] \quad (10.38)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top [\mathbf{S}_t \mathbf{X} \mathbf{w}_t + \mathbf{y} - \boldsymbol{\mu}_t] \quad (10.39)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S}_t \mathbf{z}_t \quad (10.40)$$

where we have defined the **working response** as

$$\mathbf{z}_t \triangleq \mathbf{X} \mathbf{w}_t + \mathbf{S}_t^{-1} (\mathbf{y} - \boldsymbol{\mu}_t) \quad (10.41)$$

and $\mathbf{S}_t = \text{diag}(\mu_{t,n}(1 - \mu_{t,n}))$. Since $\mathbf{S}_t$ is a diagonal matrix, we can rewrite the targets in component form as follows:

$$z_{t,n} = \mathbf{w}_t^\top \mathbf{x}_n + \frac{y_n - \mu_{t,n}}{\mu_{t,n}(1 - \mu_{t,n})} \quad (10.42)$$

Equation (10.40) is an example of a weighted least squares problem (Section 11.2.2.4), which is a minimizer of

$$\sum_{n=1}^N S_{t,n} (z_{t,n} - \mathbf{w}_t^\top \mathbf{x}_n)^2 \quad (10.43)$$

Algorithm 10.1: Iteratively reweighted least squares (IRLS)

```

1  $\mathbf{w} = \mathbf{0}$ 
2 repeat
3   for  $n = 1 : N$  do
4      $a_n = \mathbf{w}^\top \mathbf{x}_n$ 
5      $\mu_n = \sigma(a_n)$ 
6      $s_n = \mu_n(1 - \mu_n)$ 
7      $z_n = a_n + \frac{y_n - \mu_n}{s_n}$ 
8    $\mathbf{S} = \text{diag}(s_{1:N})$ 
9    $\mathbf{w} = (\mathbf{X}^\top \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S} \mathbf{z}$ 
10 until converged

```

The overall method is therefore known as the **iteratively reweighted least squares (IRLS)** algorithm, since at each iteration we solve a weighted least squares problem, where the weight matrix $\mathbf{S}_t$ changes at each iteration. See Algorithm 10.1 for some pseudocode.

Note that **Fisher scoring** is the same as IRLS except we replace the Hessian of the actual log-likelihood with its expectation, i.e., we use the Fisher information matrix (Section 4.7.2) instead of $\mathbf{H}$. Since the Fisher information matrix is independent of the data, it can be precomputed, unlike the Hessian, which must be reevaluated at every iteration. This can be faster for problems with many parameters.

10.2.7 MAP estimation

In Figure 10.4, we saw how logistic regression can overfit when there are too many parameters compared to training examples. This is a consequence of the ability of maximum likelihood to find weights that force the decision boundary to “wiggle” in just the right way so as to curve around the examples. To get this behavior, the weights often need to be set to large values. For example, in Figure 10.4, when we use degree $K = 1$, we find that the MLE for the two input weights (ignoring the bias) is

$$\hat{\mathbf{w}} = [0.51291712, 0.11866937] \quad (10.44)$$

When we use degree $K = 2$, we get

$$\hat{\mathbf{w}} = [2.27510513, 0.05970325, 11.84198867, 15.40355969, 2.51242311] \quad (10.45)$$

And when $K = 4$, we get

$$\hat{\mathbf{w}} = [-3.07813766, \dots, -59.03196044, 51.77152431, 10.25054164] \quad (10.46)$$

One way to reduce such overfitting is to prevent the weights from becoming so large. We can do this by using a zero-mean Gaussian prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, C\mathbf{I})$, and then using MAP estimation, as we discussed in Section 4.5.3. The new training objective becomes

$$\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (10.47)$$

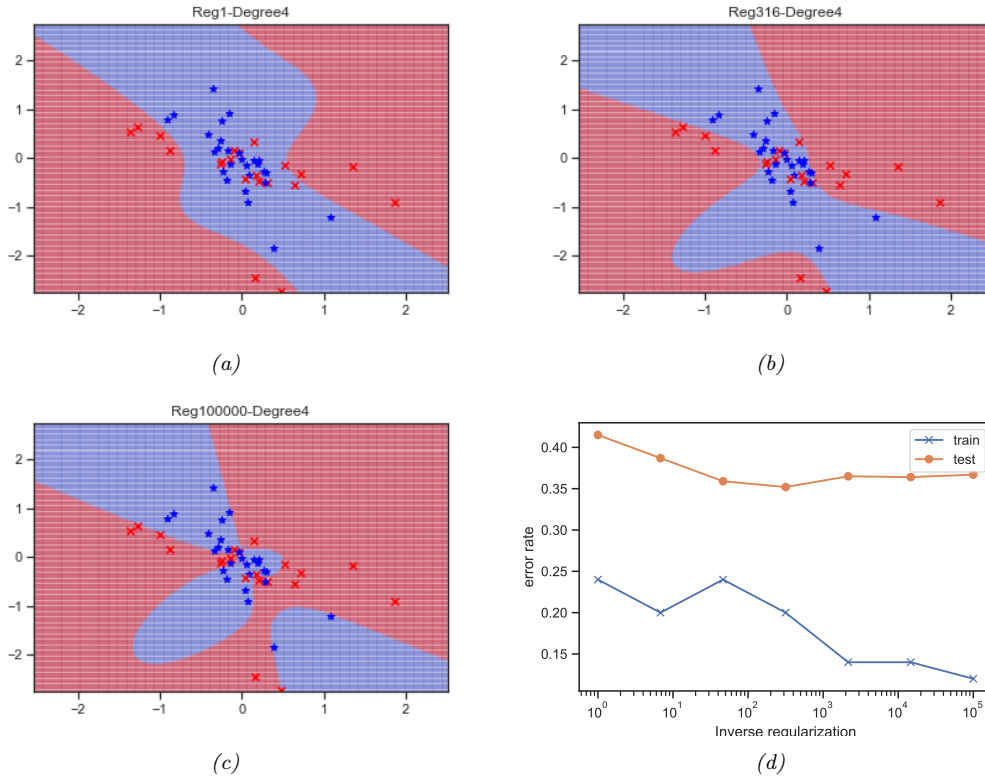


Figure 10.6: Weight decay with variance C applied to two-class, two-dimensional logistic regression problem with a degree 4 polynomial. (a) $C = 1$. (b) $C = 316$. (c) $C = 100,000$. (d) Train and test error vs C . Generated by [logreg_poly_demo.ipynb](#).

where $\|\mathbf{w}\|_2^2 = \sum_{d=1}^D w_d^2$ and $\lambda = 1/C$. This is called ℓ_2 **regularization** or **weight decay**. The larger the value of λ , the more the parameters are penalized for being “large” (deviating from the zero-mean prior), and thus the less flexible the model. See Figure 10.6 for an illustration.

We can compute the MAP estimate by slightly modifying the input to the above gradient-based optimization algorithms. The gradient and Hessian of the penalized negative log likelihood have the following forms:

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^\top \mathbf{w} \quad (10.48)$$

$$\nabla_{\mathbf{w}} \text{PNLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) + 2\lambda \mathbf{w} \quad (10.49)$$

$$\nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w}) = \mathbf{H}(\mathbf{w}) + 2\lambda \mathbf{I} \quad (10.50)$$

where $\mathbf{g}(\mathbf{w})$ is the gradient and $\mathbf{H}(\mathbf{w})$ is the Hessian of the unpenalized NLL.

For an interesting exercise related to ℓ_2 regularized logistic regression, see Exercise 10.2.

10.2.8 Standardization

In Section 10.2.7, we use an isotropic prior $\mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$ to prevent overfitting. This implicitly encodes the assumption that we expect all weights to be similar in magnitude, which in turn encodes the assumption we expect all input features to be similar in magnitude. However, in many datasets, input features are on different scales. In such cases, it is common to **standardize** the data, to ensure each feature has mean 0 and variance 1. We can do this by subtracting the mean and dividing by the standard deviation of each feature, as follows:

$$\text{standardize}(x_{nd}) = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d} \quad (10.51)$$

$$\hat{\mu}_d = \frac{1}{N} \sum_{n=1}^N x_{nd} \quad (10.52)$$

$$\hat{\sigma}_d^2 = \frac{1}{N} \sum_{n=1}^N (x_{nd} - \hat{\mu}_d)^2 \quad (10.53)$$

An alternative is to use **min-max scaling**, in which we rescale the inputs so they lie in the interval $[0, 1]$. Both methods ensure the features are comparable in magnitude, which can help with model fitting and inference, even if we don't use MAP estimation. (See Section 11.7.5 for a discussion of this point.)

10.3 Multinomial logistic regression

Multinomial logistic regression is a discriminative classification model of the following form:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y|\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})) \quad (10.54)$$

where $\mathbf{x} \in \mathbb{R}^D$ is the input vector, $y \in \{1, \dots, C\}$ is the class label, $\text{softmax}()$ is the softmax function (Section 2.5.2), $\mathbf{W}$ is a $C \times D$ weight matrix, $\mathbf{b}$ is C -dimensional bias vector, $\boldsymbol{\theta} = (\mathbf{W}, \mathbf{b})$ are all the parameters. (We will henceforth assume we have prepended each $\mathbf{x}$ with a 1, and added $\mathbf{b}$ to the first column of $\mathbf{W}$, so this simplifies to $\boldsymbol{\theta} = \mathbf{W}$.)

If we let $\mathbf{a} = \mathbf{W}\mathbf{x}$ be the C -dimensional vector of **logits**, then we can rewrite the above as follows:

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{e^{a_c}}{\sum_{c'=1}^C e^{a_{c'}}} \quad (10.55)$$

Because of the normalization condition $\sum_{c=1}^C p(y_n = c|\mathbf{x}_n, \boldsymbol{\theta}) = 1$, we can set $\mathbf{w}_C = \mathbf{0}$. (For example, in binary logistic regression, where $C = 2$, we only learn a single weight vector.) Therefore the parameters $\boldsymbol{\theta}$ correspond to a weight matrix $\mathbf{W}$ of size $(C - 1) \times D$, where $\mathbf{x}_n \in \mathbb{R}^D$.

Note that this model assumes the labels are mutually exclusive, i.e., there is only one true label. For some applications (e.g., **image tagging**), we want to predict one or more labels for an input; in this case, the output space is the set of subsets of $\{1, \dots, C\}$. This is called **multi-label classification**, as opposed to **multi-class classification**. This can be viewed as a bit vector, $\mathcal{Y} = \{0, 1\}^C$, where the c 'th output is set to 1 if the c 'th tag is present. We can tackle this using a modified version of